实 验 报 告

实验名称：实验一 Hopfield 模型的实现

课程名称：认知科学与类脑计算

实验地点：K2

实验日期：2024.9.13

班 级：2022 级人工智能班

姓 名：薛润泽

学 号：202200130077

一 实验目的：

加深对 Hopfield 模型的理解，能够使用 Hopfield 模型解决实际问题

二 实验环境：

Python, windows

三 实验内容：

根据 Hopfield 神经网络的相关知识，设计一个具有联想记忆功能的 离散型 Hopfiled 神经网络。要求该网络可以正确识别 0-9 这 10 个数字，当数字 被一定的噪声干扰后，仍具有较好的识别效果

（1） 设计 6*5 数字点阵。有数字部分用 1 表示，空白部分用-1 表示，将数字 0-9 的矩阵设计好存储到 1*30 矩阵中。

（2） 创建网络。

（3） 产生带噪声的数字点阵。带噪声的数字点阵，即点阵的某些 位置的值发生了变化。模拟产生带噪声的数字矩阵方法有很 多种，如固定噪声产生法和随机噪声产生法。

（4） 数字识别测试。将带噪声的数字点阵输入到创建好 Hopfiled 网络，网络的输出是与该数字点阵最为接近的目标向量，从 而实现联想记忆功能。

四 实验步骤：

1. 实验程序：

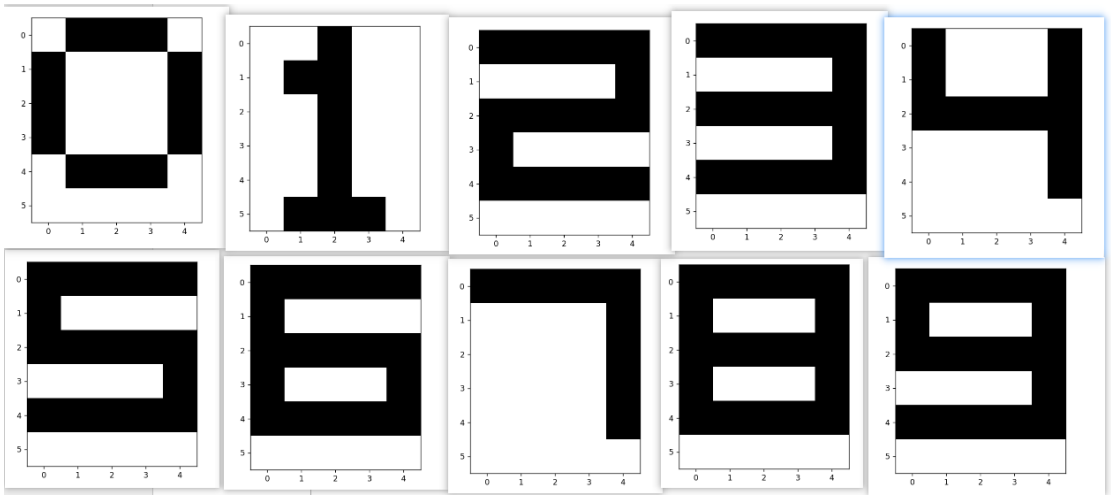
（1）实验原理：Hopfield 神经网络是一种单层神经元网络。

Hopfield 神经网络中每个神经元与其他神经元相互连接，构成单层全连接网络，并且不存在自反馈。Hopfield 神经网络具有识别和恢复图像的作用。构建 Hopfield 神经网络时，需要记录神经元与神经元之间连接的权重，同时构建出边权重矩阵。正是使用神经元与神经元之间的边权重矩阵，使得 Hopfield 神经网络能够不断修复受到噪声影响的图像，在不断的修复中达到稳定。

（2）构建出 0~9 数字图像，使用-1, 1 的 5×6 矩阵表示出 0~9

```
ori = [[[-1 ,1 ,1 ,1 ,-1],[1 ,-1 ,-1, -1, 1],[1, -1 ,-1 ,-1 ,1],[1, -1 ,-1 ,-1 ,1],[-1, 1 ,1, 1, -1],[-1, -1 ,-1 ,-1 ,-1]],
       [[-1, -1, 1, -1, -1],[-1, 1, 1, -1, -1],[-1, -1, 1, -1, -1],[-1, -1, 1, -1, -1],[-1, -1, 1, -1, -1],[-1, 1, 1, 1, -1]],
       [[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[1, -1, -1, -1, -1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]],
       [[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]],
       [[1, -1, -1, -1, 1],[1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, -1]],
       [[1, 1, 1, 1, 1],[1, -1, -1, -1, -1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]],
       [[1, 1, 1, 1, 1],[1, -1, -1, -1, -1],[1, 1, 1, 1, 1],[1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]],
       [[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, -1]],
       [[1, 1, 1, 1, 1],[1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]],
       [[1, 1, 1, 1, 1],[1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]]]
```

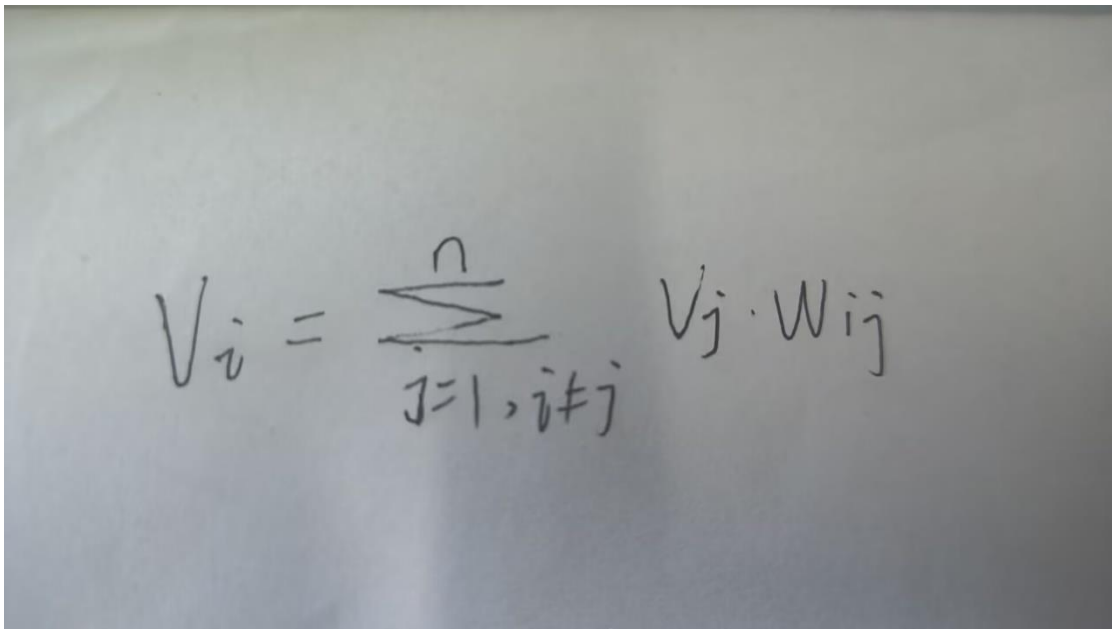
画出这十幅图



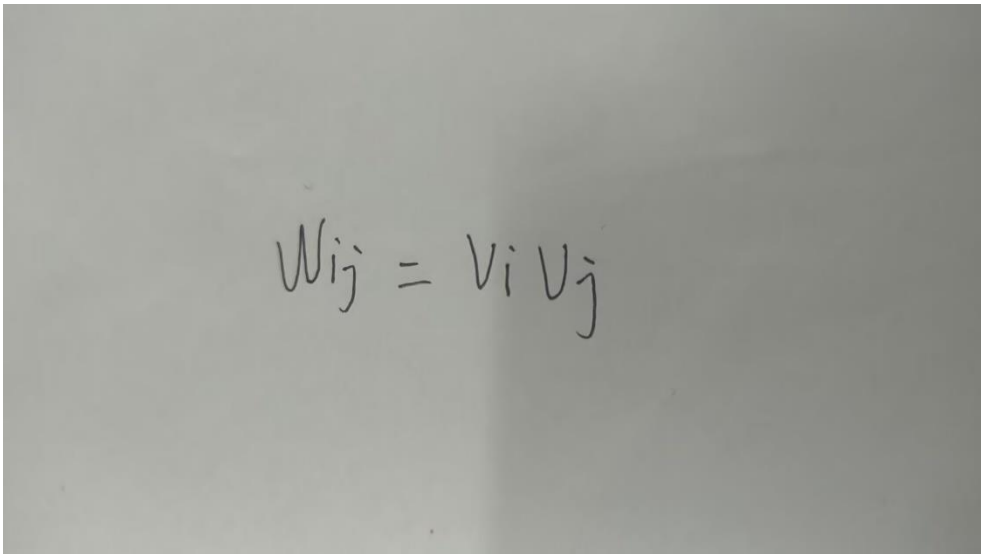
（3）Hopfield 模型的重点是构建出神经元与神经元之间的权值矩阵。

我们把数字图像中的每一个值（即着色点）看做一个神经元。因为使用的是5×6大小的矩阵来记录数字图像，所以共有30个神经元。要想让这30个神经元与每个其他神经元之间全连接，则权值矩阵w的大小为30×30（因为不存在自反馈，对角线处值为0）。

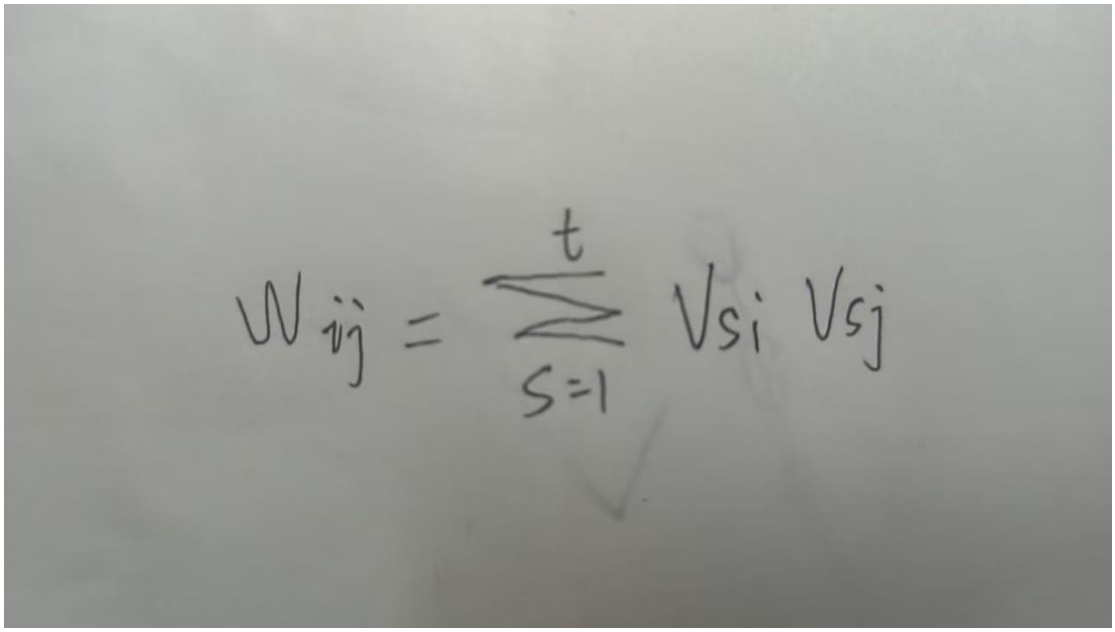
考虑如何计算权值矩阵w，可以先考虑我们有权值矩阵后应该如何修复。对于一个神经元，它的输入源自与周围神经元的输出，即如下公式：



考虑两点之间的情况，假设Vi只受到Vj的影响，因为Vj，Vi取值为1或者-1，所以可得出Wij计算式：



考虑 Wij 会由多组训练数据得出，可得 Wij 计算式：



所以构建权值矩阵需要计算所有测试数据间每组数据不同神经元的乘积和。

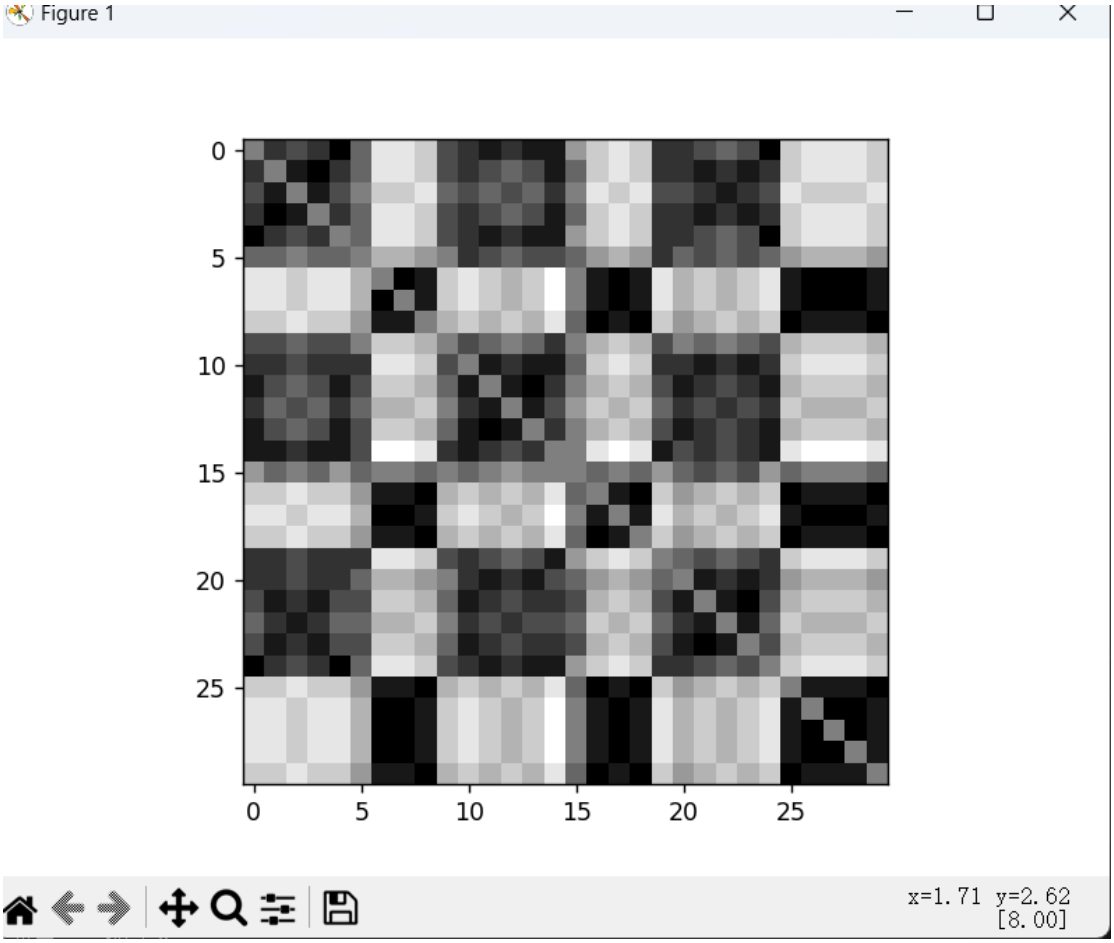
（4）Hopfield 模型检测输入数字图像以及修复数字图像

Hopfield 模型实际上是先修复数字图像然后再进行检测的。按照公式 Vi = ΣVj×Wij，将每个神经元的值进行更新，不断更新最终趋于稳定。更新过程分为逐步更新和同时更新，我选择使用同时更新的方法。将图像更新完成后检测该图像与 0-9 的每个图像相似度，选择相似度最大的图像编号进行输出。同时输出修复好的数字图像。

2. 程序实验结果：

（1）选取训练数据集为数字 0-9，共 10 个数字。

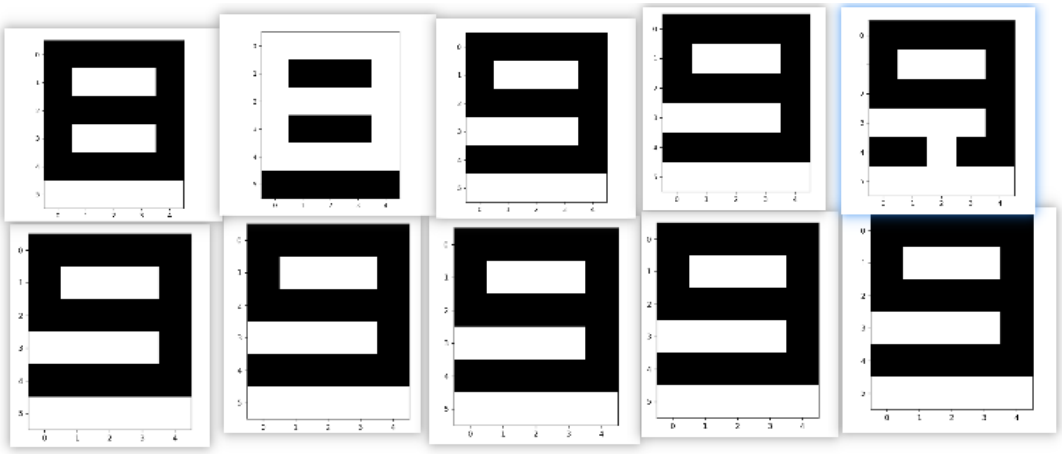
将训练出的权值矩阵画图，得到下图：



（2）首先进行 0-9,10 个不添加噪声的数字的检测。检测结果如下：

```
输入数字为：  0 检测结果为：数字 8
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  0 检测结果为：数字 8
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  1 检测结果为：数字 1
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  2 检测结果为：数字 9
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  3 检测结果为：数字 9
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  4 检测结果为：数字 9
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  5 检测结果为：数字 9
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  6 检测结果为：数字 9
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  7 检测结果为：数字 9
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  8 检测结果为：数字 9
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  9 检测结果为：数字 9
PS C:\Users\薛润泽>
```

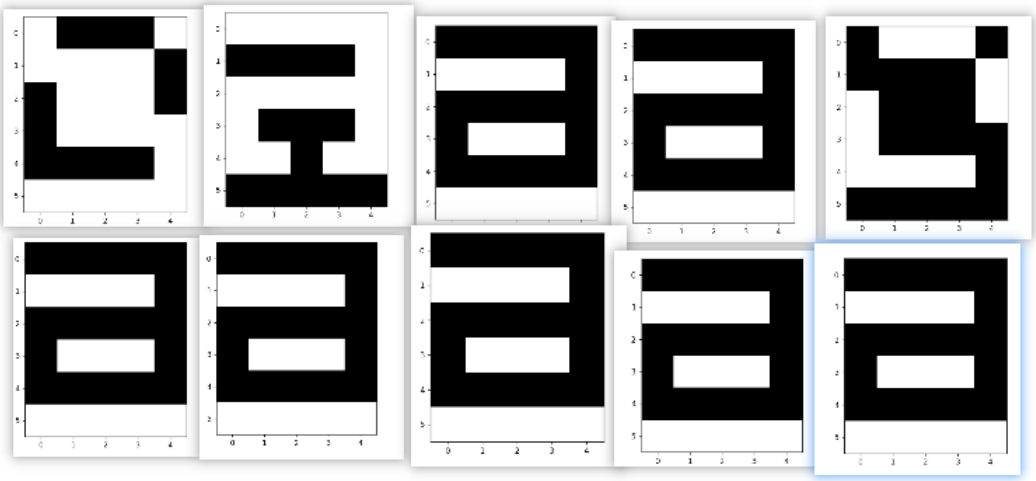
从左到右，从上到下依次为数字 0-9 的 0 噪声修复结果



可以发现，预测数字非常不准确,准确率为 20%。

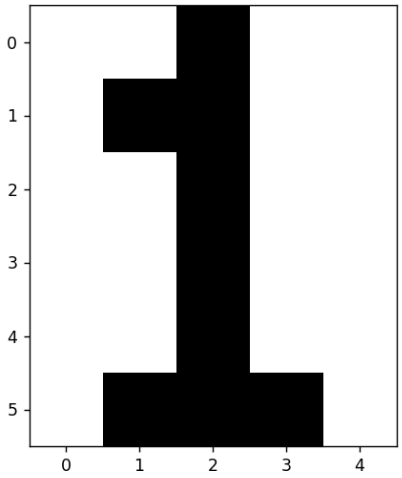
（3）降低训练样本集组数，选择数字 0-4 作为训练样本，进行无噪声预测，预测结果如下：

```
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  0 检测结果为：数字 0
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  1 检测结果为：数字 1
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  2 检测结果为：数字 2
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  3 检测结果为：数字 2
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  4 检测结果为：数字 1
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  5 检测结果为：数字 2
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  6 检测结果为：数字 2
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  7 检测结果为：数字 2
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  8 检测结果为：数字 2
PS C:\Users\薛润泽> python -u "d:\大学\大三上\认知科学与类脑识别\ex1\leinaoex1.py"
输入数字为：  9 检测结果为：数字 2
PS C:\Users\薛润泽>
```



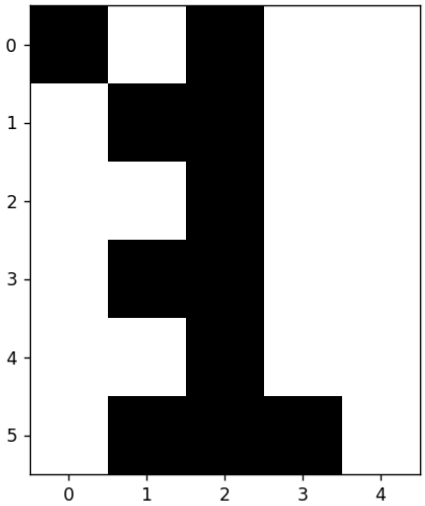
可以发现，准确率提高到 30%，对于训练的数字 0-3，准确率提高到 75%。

（5）进一步降低训练集组数，选择数字 1 进行训练，同时只对 1 无噪声数据进行预测。预测正确，且画图正确。

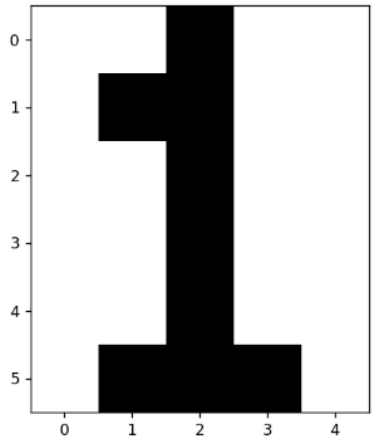


（6）对数字 1 添加随机噪声：

噪声图：



修复图：



可以发现，修复图修复完成度较高，Hopfield 模型联想效果良好。

3. 分析和改进：

当 Hopfield 模型训练数据较高时，预测效果差；当 Hopfield 模型训练数据逐渐降低到 1 组训练数据时，预测效果好，且联想效果好。

五 实验小结：

推测是因为图形数据为 5×6，导致神经元只有 30 个，过少，无法记忆过于复杂的数据。所以当训练集多时效果差，训练集少时效果好。

六 代码附录：

```
# 1 构建 1-1 个原始图像
# 2 构建权值矩阵
# 3 根据权值矩阵构建神经元，并完成原始图像的判断输入
# 4 对原始图像添加噪声
# 5 对噪声图像进行还原，并完成判断

import numpy as np
import matplotlib.pyplot as plt
ori = [[[-1 ,1 ,1 ,1 ,-1],[1 ,-1 ,-1, -1, 1],[1, -1 ,-1 ,-1 ,1],[1,
-1 ,-1 ,-1 ,1],[-1, 1 ,1, 1, -1],[-1, -1 ,-1 ,-1 ,-1]],
       [[-1, -1, 1, -1, -1],[-1, 1, 1, -1, -1],[-1, -1, 1, -1, -1],[-1, -1,
1, -1, -1],[-1, -1, 1, -1, -1],[-1, 1, 1, 1, -1]],
       [[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[1, -1, -1, -1,
-1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]],
       [[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1,
1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]],
       [[1, -1, -1, -1, 1],[1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1,
-1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, -1]],
```

```
	[[1, 1, 1, 1, 1],[1, -1, -1, -1, -1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]],
	[[1, 1, 1, 1, 1],[1, -1, -1, -1, -1],[1, 1, 1, 1, 1],[1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]],
	[[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, 1],[-1, -1, -1, -1, -1]],
	[[1, 1, 1, 1, 1],[1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]],
	[[1, 1, 1, 1, 1],[1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, 1],[1, 1, 1, 1, 1],[-1, -1, -1, -1, -1]]]

# #
def check_picture(a):
	c_id = 0
	c_num = 0
	for k in range(0, 10):
		f_num = 0
		for i in range(0, 6):
			for j in range(0, 5):
				if a[i][j] == ori[k][i][j]:
					f_num = f_num + 1
		if c_num < f_num:
			c_num = f_num
			c_id = k
	return c_id
def step_function(a):
	if a <= 0:
		a = -1
	else :
		a = 1
	return a
start_num = 1
end_num = 2
choose_num  = 1
```

```
choise_matrix = ori[choose_num]
w = np.zeros((30,30))
for k in range(start_num, end_num):
	w0 = np.zeros((30,30))
	array_ori =  [item for row in ori[k] for item in row]
	for i in range(0, 30):
		for j in  range(0, 30):
			if i != j:
				w0[i][j] +=  array_ori[i] * array_ori[j]
			else:
				w0[i][j] = 0
	w += w0

# plt.imshow(w, cmap='binary',interpolation = 'nearest')
# plt.show()

# 初始化一个全 0 的矩阵
noise_matrix = np.zeros((6, 5))

# 计算需要添加的噪声点数
noise_ratio = 0.15
num_noise = int(noise_ratio * 6 * 5)
# 随机选择噪声点的位置（行索引和列索引）
rows = np.random.choice(6, num_noise, replace=False)
cols = np.random.choice(5, num_noise, replace=False)
# 生成噪声值（-1 或 1）
noise_values = np.random.choice([-1, 1], size=num_noise)

# 将噪声值添加到选定的位置
noise_matrix[rows, cols] = noise_values
choise_matrix = choise_matrix + noise_matrix
# print(noise_matrix)
for i in range(0, 6):
	for j in range(0, 5):
		if choise_matrix[i][j] >= 2:
```

```
                choise_matrix[i][j] = 1
            elif choise_matrix[i][j] <= -2:
                choise_matrix[i][j] = -1
            elif choise_matrix[i][j] == 0:
                choise_matrix[i][j] = 1
plt.imshow(choise_matrix, cmap='binary',interpolation = 'nearest')
plt.show()
# print(choise_matrix)

result_matrix = np.zeros((6,5))
list_choise_matrix = [item for row in choise_matrix for item in row]
for i in range(0, 6):
      for j in range(0, 5):
            for k in range(start_num, end_num):
                if k != i * 6 + j:
                      result_matrix[i][j] += w[k][i * 5 + j] *
list_choise_matrix[k]
# print(choise_matrix)

for i in range(0, 6):
      for j in range(0, 5):
            result_matrix[i][j] = step_function(result_matrix[i][j])
            # if result_matrix[i][j] > 0:
            #       result_matrix[i][j] = 1
            # elif result_matrix[i][j] <= 0:
            #       result_matrix[i][j] = -1


plt.imshow(result_matrix, cmap='binary',interpolation = 'nearest')
plt.show()
c_num = check_picture(result_matrix)
print('输入数字为：  ',choose_num,'检测结果为：数字', c_num)
```